The QR Algorithm, Factor Flipping, and Inverse Iteration

Biajid Ejaj Chowdhury

June 23, 2026

1 The Big Picture

Suppose we have a real symmetric matrix $A \in \mathbb{R}^{m \times m}$. Our goal is to understand how to compute its entire spectrum of eigenvalues and eigenvectors simultaneously. An eigenpair (λ, v) satisfies the fundamental relation:

$$Av = \lambda v, \qquad v \neq 0.$$

While single eigenpairs can be isolated using methods like the standard power iteration or inverse iteration, commercial computing frameworks rely on the global convergence properties of the **QR Algorithm**. Developed independently in the late 1950s by John Francis and Vera Kublanovskaya, this approach computes the complete set of eigenpairs at once.

Our complete strategy to build a robust numerical eigensolver consists of:

1. Running the unshifted QR algorithm loop via repeated factorization and factor-flipping ($A_k = R_k Q_k$) to drive the matrix into an upper-triangular or diagonal form containing the eigenvalues.
2. Utilizing an upper-triangular convergence checker to monitor subdiagonal decay.
3. Extracting the computed eigenvalues and applying a perturbed **Inverse Iteration** routine using random starting vectors to accurately pull out the corresponding eigenvectors.

2 The Core Problem: Multi-Vector Power Method Collapse

To understand why the QR algorithm functions, we must examine what happens when we attempt to run the standard power method on multiple vectors at the same time. Let

$X_0 \in \mathbb{R}^{m \times m}$ be a matrix whose columns are random, linearly independent starting vectors. If we simply multiply this matrix by A repeatedly, we generate a sequence of matrices:

$$X_k = A^k X_0.$$

Ordering the eigenvalues by magnitude, $|\lambda_1| > |\lambda_2| > \cdots \geq |\lambda_m|$, every single column of X_k expands along the dominant eigen-direction. In the eigenvector basis expansion, the component associated with λ_1 quickly overwhelms all other components.

As $k \to \infty$, every column vector in X_k tilts toward the same dominant eigenvector v_1. Consequently, the columns become nearly identical and lose their linear independence. The matrix collapses numerically, preventing the isolation of the remaining eigenvectors.

3 The Fix: Simultaneous Iteration via QR Separation

To stop the columns from collapsing onto the same dominant vector, we must force them to remain separated. This is achieved by enforcing strict orthogonality after each multiplication step using a reduced QR factorization. This upgraded framework is called **Simultaneous Iteration**.

We initialize an orthogonal matrix $\underline{Q}_0 = I$. The loop proceeds for $k = 1, 2, \ldots$ as follows:

1. Multiply the current orthogonal frame by A to advance the power step:

$$X_k = A\underline{Q}_{k-1}$$

2. Straighten the distorted vectors back out into a perfectly perpendicular grid using a QR decomposition:

$$\underline{Q}_k \underline{R}_k = X_k$$

This step-by-step orthogonalization acts like a cascading geometric restriction:

- The first column q_1 has a free pass to align directly with the absolute dominant eigenvector v_1.
- The second column q_2 also wants to tilt toward v_1, but the QR step strips away any projection along q_1. Trapped in a perpendicular subspace, q_2 is forced to track the next best available stretching force: the second eigenvector v_2.
- Each subsequent column q_i is stripped of its projections along all prior columns, forcing the system to map out distinct, orthogonal channels corresponding to the descending spectrum of A.

As $k \to \infty$, the cumulative tracking matrix $\underline{Q}_k$ converges directly to the true eigenvector matrix V.

4 The Rayleigh Quotient Perspective

With the rotating tracking frame $\underline{Q}_k$ converging toward the true eigenvector basis, we can peek at the eigenvalues by observing A from the perspective of this shifting coordinate system. This is done via the Rayleigh quotient similarity transformation:

$$A_k = \underline{Q}_k^T A_0 \underline{Q}_k.$$

Since $\underline{Q}_k \to V$, the transformation approaches $V^T A_0 V$. For a real symmetric matrix, this maps directly to a diagonal matrix D containing the pure eigenvalues:

$$A_k \to V^T(A_0 V) = V^T(VD) = (V^T V)D = ID = D.$$

If the matrix is non-symmetric, this transformation leaves non-zero entries only above the diagonal, settling into the upper-triangular **Schur Form**. Therefore, driving the subdiagonal elements of A_k to zero signals convergence.

5 The Brilliant Shortcut: Flipping the Factors

Maintaining the historical product matrix $\underline{Q}_k = Q_1 Q_2 \dots Q_k$ is computationally cumbersome. The breakthrough of the QR algorithm is an algebraic shortcut that executes this entire multi-vector projection implicitly inside the matrix itself.

The Algebraic Derivation

Let us define the sequence by setting $A_0 = A$. In our loop, we would compute the local QR split and immediately multiply the factors in reverse order:

$$A_{k-1} = Q_k R_k \implies A_k = R_k Q_k.$$

Let us watch this unfold step-by-step for the first two iterations to see why this factor-flip is secretly calculating the global Rayleigh quotient transformation.

Iteration 1

We compute the local decomposition:

$$A_0 = Q_1 R_1.$$

Isolating R_1 by multiplying by Q_1^T from the left yields:

$$R_1 = Q_1^T A_0.$$

Now let us evaluate our flip step, $A_1 = R_1 Q_1$, by substituting this expression for R_1:

$$A_1 = (Q_1^T A_0)Q_1 = Q_1^T A_0 Q_1.$$

The single local flip has automatically generated the first Rayleigh quotient.

Iteration 2

We take the QR split of our new matrix A_1:

$$A_1 = Q_2 R_2 \implies R_2 = Q_2^T A_1.$$

We evaluate the second flip step, $A_2 = R_2 Q_2$:

$$A_2 = (Q_2^T A_1) Q_2.$$

Substituting our expression for A_1 from the first iteration into the center of this equation gives:

$$A_2 = Q_2^T (Q_1^T A_0 Q_1) Q_2.$$

Grouping the terms on the left and right using the associative property reveals:

$$A_2 = (Q_1 Q_2)^T A_0 (Q_1 Q_2).$$

The General Connection

Extending this pattern to step k, the matrix sequence satisfies:

$$A_k = (Q_1 Q_2 \dots Q_k)^T A_0 (Q_1 Q_2 \dots Q_k) = \underline{Q}_k^T A_0 \underline{Q}_k.$$

This reveals the bridge:

- The right side accumulates a pure forward power iteration chain: $(\dots((A_0 Q_1) Q_2) Q_3 \dots)$. The matrix A_0 continually operates on its own previous orthogonal transformations, dragging them into the eigenvector directions.
- The left side naturally accumulates the corresponding transposes out front: $(\dots Q_3^T (Q_2^T (Q_1^T \dots)))$. This trail of transposes balances the equations and strips away the spatial basis, forcing A_k to settle into a clean diagonal or upper-triangular eigenvalue matrix.

6 Extracting Eigenvectors via Inverse Iteration

Once the QR loop converges and the eigenvalues λ_i are extracted from the diagonal of A_k, we can isolate the individual eigenvectors using **Inverse Iteration**.

To find the eigenvector v_i corresponding to λ_i, we set up a shifted, nearly-singular linear system:

$$(A_0 - (\lambda_i + \epsilon) I) y = q_{guess},$$

where $\epsilon = 10^{-12}$ is a tiny numerical perturbation added to prevent absolute singularity errors during matrix inversion.

The Pitfall of Static Initial Guesses

If we choose a static initial guess vector, such as $q_{guess} = \mathbf{1} = [1, 1, \ldots, 1]^T$, the algorithm can fail. If the true eigenvector v_i happens to be perfectly orthogonal to the vector of ones, its component within that guess is exactly zero:

$$v_i \cdot \mathbf{1} = 0.$$

Multiplying a zero component by a large scaling factor still leaves it at zero, preventing the system from isolating that specific eigenvector.

The Robust Fix

To guarantee a non-zero projection across all possible eigenvector directions, we must choose a randomly generated vector `rand(m, 1)` as our starting point. Running two quick steps of this inverse power shift flushes out numerical noise and guarantees clean convergence to the true normalized eigenvector.

7 Complete MATLAB Implementation

Here is the complete, self-contained numerical script implementing the unshifted QR algorithm, convergence verification, and robust random-start inverse iteration.

```
function [vect, out] = qr_method(A)
    % Complete QR Eigensolver Package
    % Input:  A    = Real symmetric matrix
    % Outputs: vect = Matrix containing normalized eigenvectors as
       columns
    %          out  = Diagonal matrix containing sorted eigenvalues

    A0 = A;
    [Q, R] = qr(A);
    A1 = R * Q;
    n = 1;
    [m, ~] = size(A);

    % Step 1: Run the QR factor-flipping loop
    while ~check_ut(A1)
        A = A1;
        [Q, R] = qr(A);
        A1 = R * Q;
        n = n + 1;

        if n > 100 * m
            error('Method failed to converge within iteration limits
               !');
```

```
        end
    end

    % Step 2: Extract and sort eigenvalues
    raw_ev = diag(A1);
    [sorted_ev, ~] = sort(raw_ev, 'descend');
    vect = zeros(m, m);

    % Step 3: Run Inverse Iteration with random starts for
       eigenvectors
    for i = 1 : m
        % Initialize random starting vector to avoid orthogonal
           blindspots
        q = rand(m, 1);

        % Formulate the perturbed shifted matrix
        M = A0 - (sorted_ev(i) + 1e-12) * eye(m);

        % Run two rapid steps of inverse iteration for robust
           convergence
        q = M \ q;
        q = q / norm(q);

        q = M \ q;
        vect(:, i) = q / norm(q);
    end

    % Format the final eigenvalue output into a diagonal matrix
    out = diag(sorted_ev);
end
```

8 Convergence Verification: Checking Upper-Triangular Form

To determine when the QR algorithm loop has successfully driven the matrix into its eigenvalues, we need an automated way to monitor the decay of elements below the main diagonal. The function `check_ut` acts as our convergence criterion by inspecting the lower-triangular portion of the matrix at each step.

For an $m \times m$ matrix, the function scans down column i starting precisely at row $i + 1$ all the way to the final row m. If the absolute value of any subdiagonal element in that column exceeds our hard numerical tolerance of 10^{-9}, convergence is not yet achieved, and the function short-circuits to return false.

MATLAB Implementation

```
function out = check_ut(A)
    % Convergence checker for upper-triangular subdiagonal decay
    % Input:  A   = The current transformed matrix A_k
    % Output: out = Logical true if subdiagonal elements are within
       tolerance, else false

    [m, ~] = size(A);
    out = true;

    % Scan columns from 1 up to m - 1
    for i = 1 : m - 1
        % Look at all elements below the diagonal entry A(i,i)
        if any(abs(A(i + 1 : end, i)) > 1e-9)
            out = false;
            break;
        end
    end
end
```

Key Interpretation

- **Vectorized Subdiagonal Scans:** Instead of nested element-by-element loops, the code slices columns vertically using `A(i + 1 : end, i)`. This isolates all subdiagonal positions for that column in a single pass.
- **Numerical Tolerance vs. Strict Zero:** In practical floating-point arithmetic, subdiagonal elements rarely become exactly 0. Setting a tolerance threshold ($\epsilon = 10^{-9}$) ensures the algorithm terminates gracefully when the values have decayed far enough to guarantee precision.
- **Early Break Optimization:** The moment a single element fails the threshold, the function exits immediately via the `break` statement, protecting the computational budget from unnecessary memory reads on large matrices.

9 Important Facts

1. Plain multi-vector power iteration fails because every column vector collapses onto the single dominant eigenvector axis.
2. Simultaneous iteration fixes this by executing a QR factorization at each step, forcing the tracking vectors to stay perfectly perpendicular.

3. The QR loop shortcut (`A = R * Q`) implicitly computes the global Rayleigh transformation without tracking historical matrix frames.

4. The right-hand factor product drives the system forward into the eigen-directions, while the left-hand transpose components undo the spatial rotation.

5. Inverse iteration requires a randomized initial guess vector (`rand(m,1)`) because a static guess can land in a mathematical blind spot orthogonal to the true eigenvector.

References

- Laurene V. Fausett, *Applied Numerical Analysis Using MATLAB*, 2nd Edition. Prentice Hall, 2008.

- Chris Rycroft, *Harvard AM205 Video 5.6 - QR algorithm.* Available at: `https://youtu.be/TtPU4KOaQ9A`